

Universidade Estadual do Ceará

Curso de Ciência da Computação

Disciplina: Processamento de Imagens

Atividade 4 — Relatório

Aluno: Samuel William Silva Almeida

Professor: Matheus Araújo

Data: Junho de 2026

1. Introdução

Esta atividade tem como objetivo introduzir técnicas fundamentais de morfologia matemática e segmentação de imagens, por meio da implementação prática de operadores que permitem extrair, separar e estruturar informações relevantes em imagens digitais. Foram implementados três conjuntos de algoritmos: operações morfológicas (erosão, dilatação, abertura e fechamento), detecção de bordas com as primeiras etapas do algoritmo de Canny juntamente com o descritor HOG simplificado, e segmentação por watershed baseado em marcadores.

Todas as implementações foram feitas manualmente em Python, utilizando as bibliotecas OpenCV e NumPy. O OpenCV foi utilizado exclusivamente para carregamento e salvamento de imagens, enquanto o NumPy foi empregado para as operações computacionais. O tema central das imagens utilizadas é o universo da moda e vestuário: temos fotos de ternos, vestidos, modelos e pilhas de roupas.

2. Questão 1 — Operações Morfológicas

Foram implementadas as operações morfológicas de erosão e dilatação em imagens binárias obtidas a partir de duas imagens do tema (terno e vestido). Utilizaram-se elementos estruturantes quadrados de três tamanhos distintos: 3×3 , 5×5 e 15×15 . A partir dessas operações primitivas, construíram-se também as operações de abertura (erosão seguida de dilatação) e fechamento (dilatação seguida de erosão).

2.1 Implementação

```
def criar_ee(tamanho):
    return np.ones((tamanho, tamanho), dtype=np.uint8)

def erosao(img, ee):
    for y in range(mh, h - mh):
        for x in range(ml, w - ml):
            if np.min(img[y-mh:y-mh+eh, x-ml:x-ml+el]) == 255:
                out[y, x] = 255

def dilatacao(img, ee):
    for y in range(mh, h - mh):
        for x in range(ml, w - ml):
            if np.max(img[y-mh:y-mh+eh, x-ml:x-ml+el]) == 255:
                out[y, x] = 255

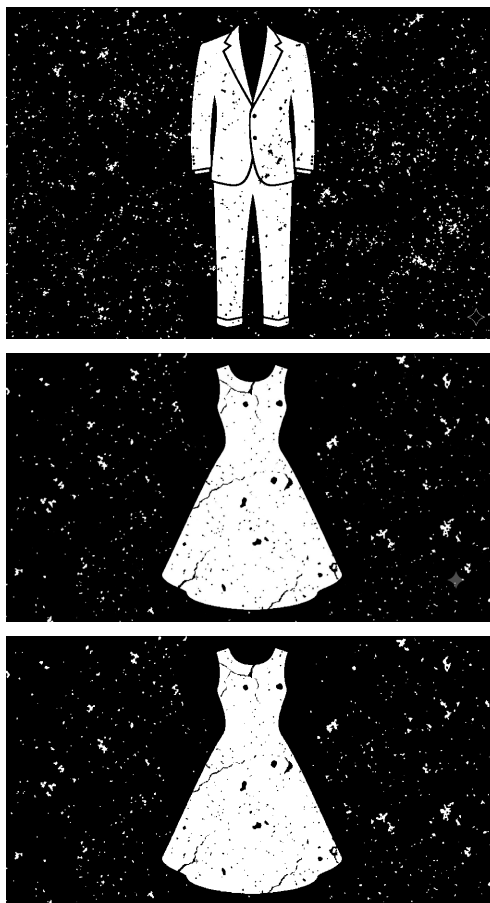
def abertura(img, ee):
    return dilatacao(erosao(img, ee), ee)

def fechamento(img, ee):
    return erosao(dilatacao(img, ee), ee)
```

2.2 Imagens originais e binarizadas

As imagens do terno e do vestido foram convertidas para tons de cinza utilizando a fórmula de luminância ITU-R BT.601 ($0,299R + 0,587G + 0,114B$) e binarizadas com limiar 127.

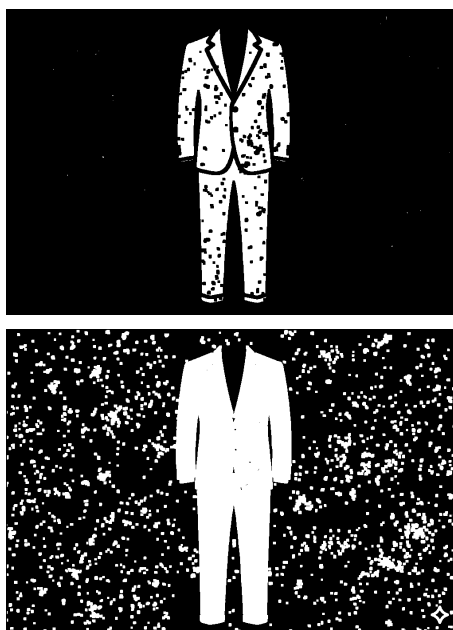


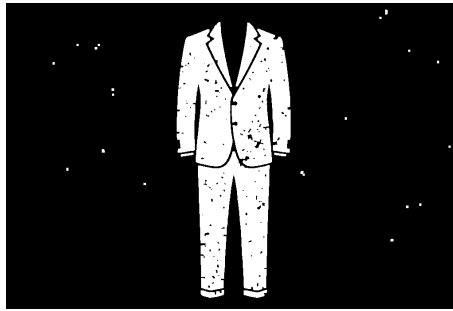


2.3 Resultados — Erosão, Dilatação, Abertura e Fechamento

As figuras a seguir apresentam os resultados para cada operação com elementos estruturantes de 3×3 , 5×5 e 15×15 . Observa-se que, com EE 3×3 , as alterações são sutis; com EE 5×5 , os efeitos tornam-se mais pronunciados; já com EE 15×15 , ocorre transformação significativa.

Resultados — Terno





Resultados — Vestido



Análise: A erosão reduz o tamanho dos objetos, eliminando pequenas projeções e ruídos. A dilatação expande os objetos, preenchendo pequenos buracos. A abertura suaviza contornos e remove protuberâncias finas. O fechamento preenche pequenos laços e une regiões próximas. A escolha do

tamanho do EE depende da escala dos detalhes que se deseja preservar ou remover.

3. Questão 2 — Filtros, Detecção de Bordas (Canny) e HOG

Implementaram-se as duas primeiras etapas do algoritmo de detecção de bordas de Canny: (1) suavização por filtro gaussiano com kernel 5×5 e $\sigma = 1,4$ e (2) cálculo do gradiente utilizando operadores de Sobel 3×3 implementados manualmente. Como extensão, implementou-se o descritor HOG simplificado com células de 8×8 pixels e 9 bins de orientação.

3.1 Implementação

```
def gaussian_kernel_2d(kernel_size, sigma):
    kernel = np.zeros((kernel_size, kernel_size))
    center = kernel_size // 2
    for i in range(kernel_size):
        for j in range(kernel_size):
            dx, dy = i - center, j - center
            kernel[i,j] = exp(-(dx**2+dy**2)/(2*sigma**2)) / (2*pi*sigma**2)
    return kernel / kernel.sum()

def convolve_2d(image, kernel):
    padded = np.pad(image, pad_width=..., mode="edge")
    for i in range(height):
        for j in range(width):
            output[i,j] = np.sum(padded[i:i+kh, j:j+kw] * kernel)

def sobel_gradient(image):
    gx = convolve_2d(image, [[-1,0,1],[-2,0,2],[-1,0,1]])
    gy = convolve_2d(image, [[-1,-2,-1],[0,0,0],[1,2,1]])
    return np.sqrt(gx**2 + gy**2), np.arctan2(gy, gx)
```

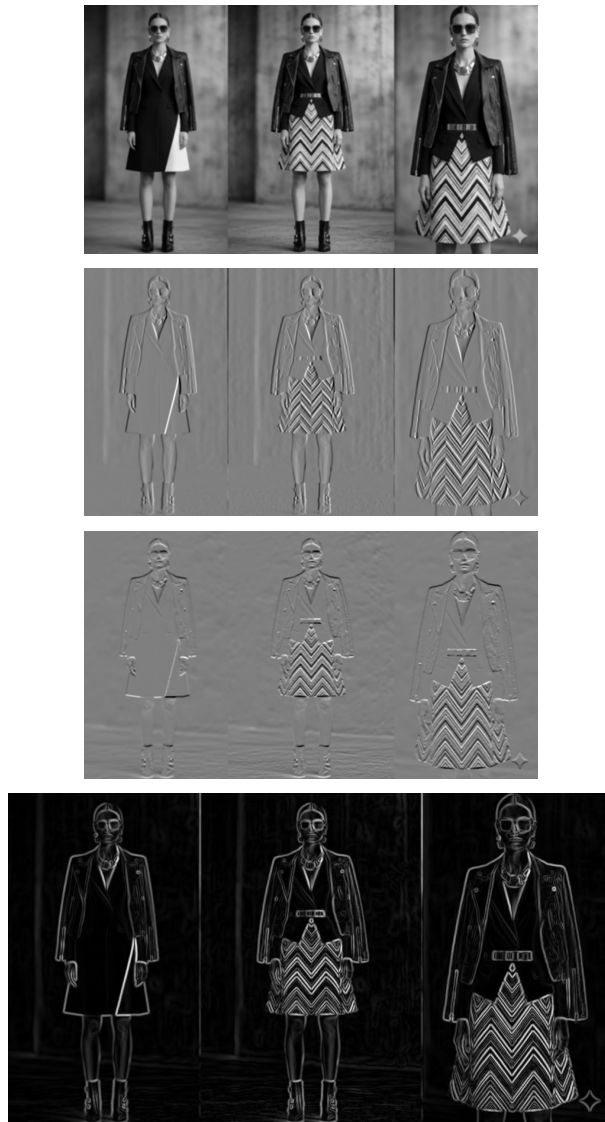
3.2 Imagem de entrada

A imagem utilizada foi a de um modelo (2_modelo.png), relacionada ao tema moda e vestuário.



3.3 Filtro Gaussiano e Gradientes

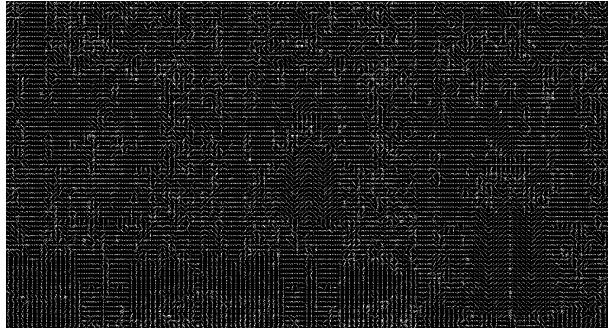
O filtro gaussiano suaviza a imagem, reduzindo ruídos antes do cálculo do gradiente. As componentes do gradiente nos eixos X e Y foram calculadas com os operadores Sobel. A magnitude do gradiente evidencia as bordas.



3.4 Descritor HOG

O HOG divide a imagem em células de 8×8 pixels. Para cada pixel, o ângulo de orientação do gradiente é quantizado em um dos 9 bins (0° – 180°) e o histograma é ponderado pela magnitude do gradiente. A visualização abaixo mostra linhas proporcionais ao valor de cada bin em cada célula.

```
def hog_descriptor(magnitude, orientation, cell_size=8, nbins=9):
    for each cell:
        hist = zeros(nbins)
        for each pixel in cell:
            bin = quantize(orientation[pixel], nbins)
            hist[bin] += magnitude[pixel]
        cell_histograms.append(hist)
    return cell_histograms
```



O vetor de características gerado possui 69×127 células $\times$ 9 bins = aproximadamente 79 mil features. Cada feature representa a contribuição de uma orientação específica em uma região local, capturando a estrutura das bordas da imagem de forma compacta.

4. Questão 3 — Segmentação Watershed

Implementou-se um método de segmentação baseado em marcadores seguido da aplicação do algoritmo Watershed, com o objetivo de separar objetos em regiões próximas ou parcialmente sobrepostas. A imagem utilizada foi a de chapéus empilhados (3_chapeus.png).

4.1 Pipeline

O pipeline segue as etapas: (1) limiarização (threshold=127) para binarização; (2) fechamento morfológico com kernel 5x5 para limpeza de ruídos; (3) distance transform de Chamfer 3-4-5 em dois passes para gerar o mapa de distâncias; (4) identificação de máximos locais (acima de 85% do valor máximo) como marcadores de primeiro plano; (5) watershed por inundação com priority queue ordenada por distância decrescente; (6) detecção das linhas de fronteira entre segmentos adjacentes.

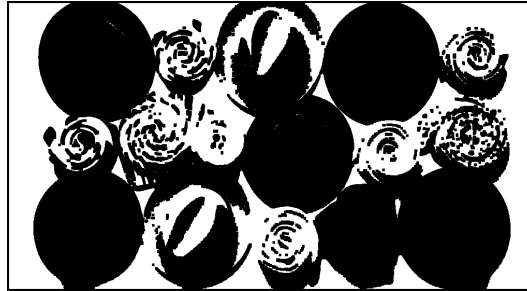
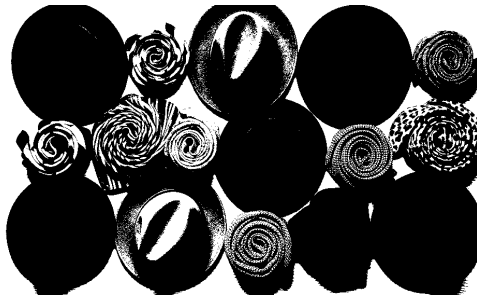
4.2 Implementação

```
def distance_transform_chamfer(binary):
    # Forward pass: top-left to bottom-right
    for y in range(height):
        for x in range(width):
            d[y,x] = min(d[y,x], d[y-1,x-1]+4, d[y-1,x]+3,
                        d[y-1,x+1]+4, d[y,x-1]+3)
    # Backward pass: bottom-right to top-left
    for y in reversed range(height):
        for x in reversed range(width):
            d[y,x] = min(d[y,x], d[y+1,x-1]+4, d[y+1,x]+3,
                        d[y+1,x+1]+4, d[y,x+1]+3)
    return d / 3.0

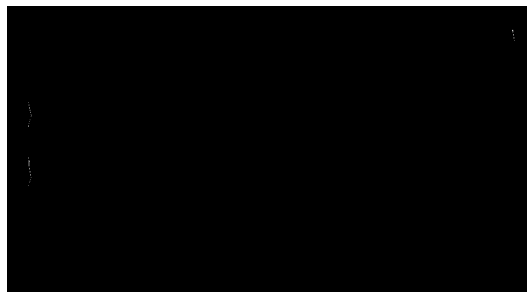
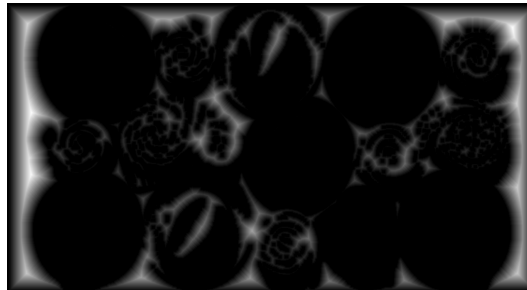
def watershed(distance, markers):
    heap = [] # priority queue by -distance
    for each marker pixel: push to heap
    while heap:
        _, y, x = heappop(heap)
        label = labels[y,x]
        for neighbor (4-dir):
            if unlabeled and distance > 0:
                labels[ny,nx] = label
                heappush(heap, (-distance[ny,nx], ny, nx))
```

4.3 Etapas intermediárias

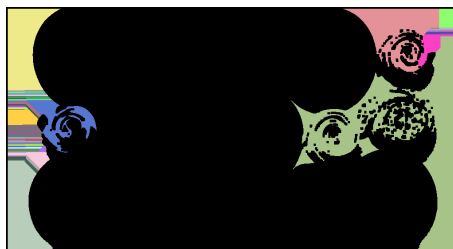
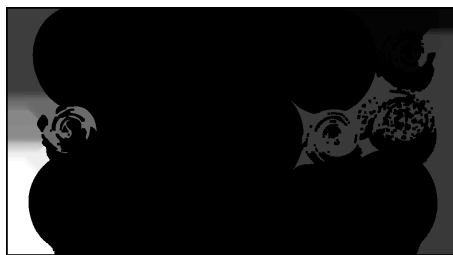




Distance transform (Chamfer) e marcadores (máximos locais):



4.4 Resultado da segmentação



Foram identificados 62 segmentos na imagem de chapéus. O algoritmo conseguiu separar adequadamente os chapéus sobrepostos. A etapa de fechamento morfológico foi fundamental para reduzir ruídos que causariam super-segmentação. O limiar de 85% do valor máximo do distance transform na seleção de marcadores mostrou-se adequado para identificar os centros dos objetos com poucos marcadores espúrios.

5. Conclusão

A atividade 4 de Processamento de Imagens permitiu a implementação prática de técnicas fundamentais de morfologia matemática, detecção de bordas e segmentação de imagens. As operações morfológicas demonstraram como diferentes tamanhos de elemento estruturante afetam a forma de objetos binários. O filtro gaussiano e o gradiente Sobel constituem as bases do detector de bordas de Canny, enquanto o HOG mostrou-se capaz de capturar a estrutura direcional das bordas. A segmentação watershed revelou-se eficaz para separar objetos adjacentes, desde que o pré-processamento seja adequado.

Todas as implementações foram realizadas manualmente conforme as regras da atividade, utilizando OpenCV exclusivamente para carregamento e salvamento de imagens, e NumPy para as operações de cálculo e álgebra linear.